

ADS Codes

Binary Tree:

```
class Node:

    def __init__(s,v): s.v,v; s.l=s.r=None

from collections import deque

def insert(root, v):

    if not root: return Node(v)

    q=deque([root])

    while q:

        n=q.popleft()

        if not n.l: n.l=Node(v); break

        else: q.append(n.l)

        if not n.r: n.r=Node(v); break

        else: q.append(n.r)

    return root

def inorder(t):

    if t: inorder(t.l); print(t.v,end=" "); inorder(t.r)

def preorder(t):

    if t: print(t.v,end=" "); preorder(t.l); preorder(t.r)

def postorder(t):

    if t: postorder(t.l); postorder(t.r); print(t.v,end=" ")

def search(t,x):

    if not t: return False

    return t.v==x or search(t.l,x) or search(t.r,x)

#Input

arr=list(map(int,input("Enter values: ").split()))

root=None

for x in arr: root=insert(root,x)

print("Inorder :",end=" "); inorder(root); print()
```

```
print("Preorder :",end=" "); preorder(root); print()
print("Postorder:",end=" "); postorder(root); print()
k=int(input("Search: "))
print("FOUND" if search(root,k) else "NOT FOUND")
```

Binary Search Tree:

```
class Node:
```

```
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None
```

```
# Insert into BST
```

```
def insert(root, key):
    if root is None:
        return Node(key)
    if key < root.key:
        root.left = insert(root.left, key)
    else:
        root.right = insert(root.right, key)
    return root
```

```
# Traversals
```

```
def inorder(root):
    if root:
        inorder(root.left)
        print(root.key, end=" ")
```

```
def preorder(root):
    if root:
        print(root.key, end=" ")
        preorder(root.left)
        preorder(root.right)
```

```

def postorder(root):
    if root:
        postorder(root.left)
        postorder(root.right)
        print(root.key, end=" ")

# Search in BST
def search(root, key):
    if root is None:
        return False
    if key == root.key:
        return True
    if key < root.key:
        return search(root.left, key)
    return search(root.right, key)

#Input
n = int(input("How many values to insert in BST? "))
print("Enter the values:")

#Input
arr = list(map(int, input().split()))
root = None
for val in arr:
    root = insert(root, val)
print("\n--- BST Traversals ---")
print("Inorder  :", end=" "); inorder(root); print()
print("Preorder :", end=" "); preorder(root); print()
print("Postorder :", end=" "); postorder(root); print()

# Search part
key = int(input("\nEnter a value to search: "))
if search(root, key):

```

```
    print(key, "FOUND in BST")
```

```
else: print(key, "NOT FOUND in BST")
```

AVL Tree:

```
class Node:
```

```
    def __init__(s,v): s.v=v; s.l=s.r=None; s.h=1
```

```
    def h(n): return n.h if n else 0
```

```
    def bf(n): return h(n.l)-h(n.r)
```

```
    def rotR(y):
```

```
        x=y.l; t=x.r
```

```
        x.r=y; y.l=t
```

```
        y.h=1+max(h(y.l),h(y.r))
```

```
        x.h=1+max(h(x.l),h(x.r))
```

```
        return x
```

```
    def rotL(x):
```

```
        y=x.r; t=y.l
```

```
        y.l=x; x.r=t
```

```
        x.h=1+max(h(x.l),h(x.r))
```

```
        y.h=1+max(h(y.l),h(y.r))
```

```
        return y
```

```
    def insert(n,v):
```

```
        if not n: return Node(v)
```

```
        if v<n.v: n.l=insert(n.l,v)
```

```
        else: n.r=insert(n.r,v)
```

```
        n.h=1+max(h(n.l),h(n.r))
```

```
        b=bf(n)
```

```
        if b>1 and v<n.l.v: return rotR(n)    # LL
```

```
        if b<-1 and v>n.r.v: return rotL(n)    # RR
```

```
        if b>1 and v>n.l.v: n.l=rotL(n.l); return rotR(n) # LR
```

```
        if b<-1 and v<n.r.v: n.r=rotR(n.r); return rotL(n) # RL
```

```
        return n
```

```
def inorder(t):
    if t: inorder(t.l); print(t.v,end=" "); inorder(t.r)

def preorder(t):
    if t: print(t.v,end=" "); preorder(t.l); preorder(t.r)
```

#Input

```
arr=list(map(int,input("Enter values: ").split()))

root=None

for x in arr:
    root=insert(root,x)

print("Inorder :",end=" "); inorder(root); print()

print("Preorder:",end=" "); preorder(root); print()
```

B-Tree:

```
class BTree:

    def __init__(s,t): s.t=t; s.root=["leaf",[],[]] # [type,keys,children]

    def search(s,x,n=None):
        n=n or s.root
        i=0
        while i<len(n[1]) and x>n[1][i]: i+=1
        if i<len(n[1]) and x==n[1][i]: return True
        return False if n[0]=="leaf" else s.search(x,n[2][i])

    def split(s,p,i):
        t=s.t; y=p[2][i]
        z=["leaf"==y[0] and "leaf" or "node", y[1][t:], y[2][t:]]
        p[1].insert(i,y[1][t-1])
        p[2].insert(i,z)
        y[1],y[2]=y[1][:t-1],y[2][:t]

    def insert(s,x):
        r=s.root
        if len(r[1])==2*s.t-1:
```

```

        s.root=["node",[],[r]]

        s.split(s.root,0)

    s._ins(s.root,x)
def _ins(s,n,x):
    if n[0]=="leaf":
        i=0

        while i<len(n[1]) and x>n[1][i]: i+=1

        n[1].insert(i,x)
    else:
        i=0

        while i<len(n[1]) and x>n[1][i]: i+=1

        if len(n[2][i][1])==2*s.t-1:
            s.split(n,i)

            if x>n[1][i]: i+=1

        s._ins(n[2][i],x)

```

INPUT

```

t=2          # minimum degree

bt=BTree(t)

vals=list(map(int,input("Enter values: ").split()))

for x in vals: bt.insert(x)

k=int(input("Search: "))

print("FOUND" if bt.search(k) else "NOT FOUND")

```

Red-Black Tree:

```

R = "RED"; B = "BLACK"

class Node:

    def __init__(s,v): s.v=v; s.l=s.r=s.p=None; s.c=R

def rotL(t,x):

    y=x.r; x.r=y.l

    if y.l: y.l.p=x

```

```

y.p=x.p

if not x.p: t.root=y

elif x==x.p.l: x.p.l=y

else: x.p.r=y

y.l=x; x.p=y

def rotR(t,x):

    y=x.l; x.l=y.r

    if y.r: y.r.p=x

    y.p=x.p

    if not x.p: t.root=y

    elif x==x.p.r: x.p.r=y

    else: x.p.l=y

    y.r=x; x.p=y

class RBT:

    def __init__(s): s.root=None

    def insert(s,v):

        n=Node(v)

        y=None; x=s.root

        while x:

            y=x

            x = x.l if n.v < x.v else x.r

        n.p=y

        if not y: s.root=n

        elif n.v < y.v: y.l=n

        else: y.r=n

        s.fix(n)

    def fix(s,n):

        while n.p and n.p.c==R:

            if n.p==n.p.p.l:

                u=n.p.p.r

```

```

if u and u.c==R:

    n.p.c=u.c=B; n.p.p.c=R; n=n.p.p

else:

    if n==n.p.r: n=n.p; rotL(s,n)

    n.p.c=B; n.p.p.c=R; rotR(s,n.p.p)

```

```

else:

    u=n.p.p.l

    if u and u.c==R:

        n.p.c=u.c=B; n.p.p.c=R; n=n.p.p

    else:

        if n==n.p.l: n=n.p; rotR(s,n)

        n.p.c=B; n.p.p.c=R; rotL(s,n.p.p)

```

```

s.root.c=B

```

```

def inorder(s,n):

```

```

    if n: s.inorder(n.l); print(n.v,end=" "); s.inorder(n.r)

```

INPUT

```

rbt = RBT()

```

```

vals = list(map(int, input("Enter values: ").split()))

```

```

for x in vals: rbt.insert(x)

```

```

print("Inorder:", end=" "); rbt.inorder(rbt.root); print()

```